

SMS Spam Detection

Классификация SMS-сообщений на spam и ham

Студент: Gleb Sanin

Локальный деплой: FastAPI + Streamlit

В отчёте собраны данные, обработка, эксперименты, выбор финальной модели и запуск сервиса.

1. Постановка задачи

Я решал задачу антиспам-фильтра для коротких SMS: по тексту сообщения нужно вернуть класс ham или spam. Общая ассигурация здесь не главный ориентир: если модель почти всегда говорит ham, она всё равно может выглядеть неплохо из-за дисбаланса классов.

Основная метрика - F1 по классу spam. Она показывает баланс между пропущенным спамом и ложной блокировкой нормальных сообщений. Дополнительно я смотрел precision, recall и ассигурацию.

2. Данные

Используется SMS Spam Collection: открытый корпус сообщений для задачи spam filtering. После удаления точных дублей осталось 5 158 сообщений.

Класс	Количество
ham	4 516
spam	642

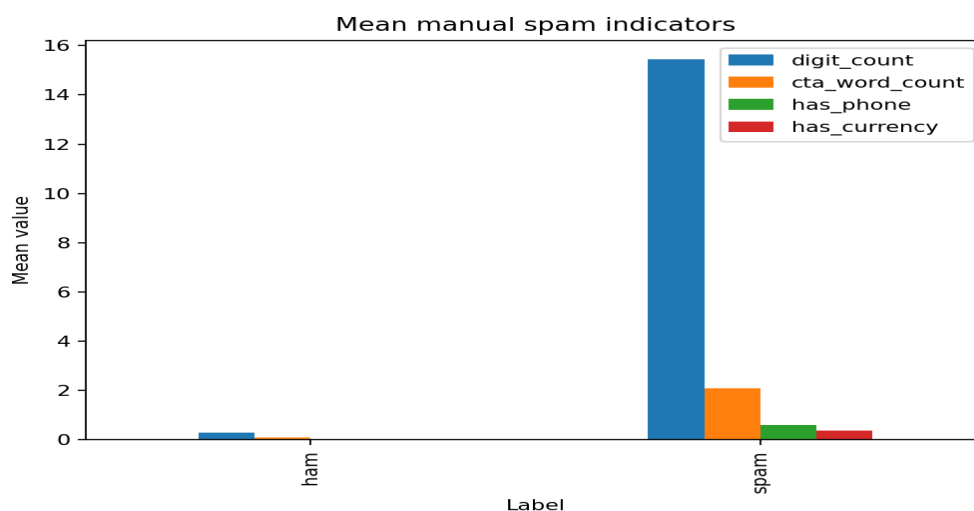
Датасет меньше формального порога в 10 000 строк, но был согласован для проекта. В сыром виде в нём две полезные колонки, зато после TF-IDF текст превращается в набор n-грамм. Дополнительно используются ручные признаки по структуре SMS.

3. Подготовка данных

- чтение исходного архива sms_spam_collection.zip;
- кодирование меток: ham = 0, spam = 1;
- удаление пустых сообщений и точных дублей;
- добавление признаков: длина, цифры, uppercase, пунктуация, URL, телефон, валюта, СТА-слова;
- стратифицированный split 70/15/15 на train, validation и test.

Векторизаторы находятся внутри sklearn pipeline, поэтому словарь и IDF считаются только по train. Validation и test не участвуют в построении признакового пространства.





4. Baseline

Первый baseline - CountVectorizer + MultinomialNB. Это быстрый и понятный вариант для текста без ручного feature engineering.

Модель	Accuracy	Precision spam	Recall spam	F1 spam
CountVectorizer + MultinomialNB	0.979	0.966	0.866	0.913

Baseline оказался сильным: многие spam-сообщения в корпусе шаблонные и содержат слова вроде free, call, claim, а также короткие номера и суммы.

5. Эксперименты

Гипотеза	Проверка	Результат
Bag-of-words даст сильный baseline	CountVectorizer + MultinomialNB	F1_spam = 0.913 на test
TF-IDF будет устойчивее	TF-IDF + LogisticRegression	F1_spam = 0.819, recall ниже
LinearSVC лучше разделит n-граммы	TF-IDF + LinearSVC	F1_spam = 0.951 на validation
Ручные признаки добавляют сигнал	TF-IDF + manual features + LogisticRegression	F1_spam = 0.928 на test

Модель	Split	Accuracy	Precision	Recall	F1
CountVectorizer + MultinomialNB	test	0.979	0.966	0.866	0.913
TF-IDF + LogisticRegression	test	0.961	0.986	0.701	0.819
TF-IDF + LinearSVC	val	0.988	1.000	0.906	0.951
TF-IDF + LinearSVC	test	0.978	0.955	0.866	0.908
TF-IDF + manual + LogisticRegression	test	0.983	1.000	0.866	0.928

6. Финальная модель

Для сервиса выбрана TF-IDF + ручные признаки + LogisticRegression. LinearSVC был сильнее на validation, но логистическая регрессия возвращает вероятность spam-класса через predict_proba. Это удобно для API и интерфейса: пользователь видит не только класс, но и spam_score.

- free, call, claim, prize, urgent двигают предсказание в сторону spam;
- телефоны, валютные символы и СТА-слова усиливают этот сигнал;
- обычные бытовые SMS без цифр, ссылок и призывов чаще остаются ham.

7. Деплой

Для CP3 сделаны два локальных способа использования модели: FastAPI для программного вызова и Streamlit для ручной проверки сообщения.

Компонент	Что делает
GET /health	Проверяет, что API поднят
POST /predict	Принимает JSON с message и возвращает label, target, spam_score
Streamlit UI	Даёт поле ввода SMS и показывает результат модели

Пример ответа API:

```
{"label": "spam", "target": 1, "spam_score": 0.9902}
```

FastAPI /predict

```
POST /predict
{"message": "URGENT! You won a free prize. Call now to claim cash."}

Response
{
  "label": "spam",
  "target": 1,
  "spam_score": 0.9902
}
```

Streamlit UI

SMS Spam Detection

Input: Hey, are we still meeting near the station at 7?
Prediction: ham
Spam score: 0.0093

Input: URGENT! You won a free prize. Call now to claim cash.
Prediction: spam
Spam score: 0.9902

Видео демонстрации

https://disk.yandex.ru/i/pJhz8z5Rq__DAw

Команды запуска:

```
uvicorn app.api:app --host 127.0.0.1 --port 8000  
streamlit run app/streamlit_app.py --server.address 127.0.0.1 --server.port 8501
```

8. Выводы

Проект можно запустить локально и проверить на своих SMS. API подходит для программного доступа, Streamlit - для ручной проверки одного сообщения.

Главное ограничение - датасет небольшой и не новый. Для реального антиспам-фильтра следующим шагом был бы сбор свежих сообщений, проверка качества на новых данных и подбор порога `spam_score` под допустимый уровень ложных срабатываний.